

Software Requirements and Design Document

For

Group <1>

Version 3.0

Authors:

Annika Needham

Isabella Maldur

Nikki Zahedi

Jaliah Meade

1. Overview (5 points)

ShiftSync is a web-based employee scheduling app designed to help businesses organize employee shift information to improve the workflow of employees and administrators. The system supports 2 user roles: employees and administrators. Employees can log in, create/enter passwords, view their dashboard, review upcoming shifts, manage their weekly availability, and submit shift swap requests. They have a dashboard upon logging that shows any upcoming shifts, a weekly calendar view, and outgoing/incoming shift swap requests. Employee accounts support hashed password storage, password creation, and a login attempt limit/reset password capability. Administrators can log in through a separate workflow and access an admin dashboard. They can access an admin dashboard with the ability to view all employee schedules and availability, and add or edit shifts for employees.

2. Functional Requirements (10 points)

List the **functional requirements** in sentences identified by numbers and for each requirement state if it is of high, medium, or low priority. Each functional requirement is something that the system shall do. Include all the details required such that there can be no misinterpretations of the requirements when read. Be very specific about what the system needs to do (not how, just what). You may provide a brief design rationale for any requirement which you feel requires explanation for how and/or why the requirement was derived.

FR1 (High Priority): The system shall allow an employee to enter an employee ID and be validated against stored employee records before accessing their account features.

FR2 (High Priority): The system shall allow a first-time employee user without a password to create a password and save it for future logins.

FR3 (High Priority): The system shall allow an existing employee with a password to use that password before accessing the employee dashboard.

FR4 (High Priority): The system shall limit employee password login attempts to three failed attempts per session and make them reset their password if all attempts failed.

FR5 (High Priority): The system shall allow an administrator to log in through a separate administrator login workflow using an admin ID and password.

FR6 (High Priority): The system shall allow authenticated users to log out and clear active session data.

FR7 (High Priority): The system shall display an employee dashboard containing the employee's name or ID, upcoming shifts, availability summary, swap request information, and weekly schedule view.

FR8 (High Priority): The system shall allow an employee to add availability blocks by selecting a day of the week, start time, and end time.

FR9 (High Priority): The system shall allow an employee to edit an existing availability block that belongs to them.

FR10 (High Priority): The system shall allow an employee to delete an existing availability block that belongs to them.

FR11 (Medium Priority): The system shall validate availability input so that only valid days and allowed time selections are accepted and end time must be later than start time.

FR12 (High Priority): The system shall allow an employee to submit a shift swap request for one of the employee's shifts.

FR13 (Medium Priority): The system shall allow an employee to optionally pick an employee ID when submitting a shift swap request and optionally include a reason.

FR14 (Medium Priority): The system shall prevent duplicate pending shift swap requests for the same requester and shift combination.

FR15 (Medium Priority): The system shall allow an employee to view submitted shift swap requests and their statuses.

FR16 (Medium Priority): The system shall allow an employee to cancel a pending shift swap request they submitted.

FR17 (High Priority): The system shall notify an employee of shift swap requests that are pending for that employee.

FR18 (High Priority): The system shall allow a targeted employee to approve a pending shift swap request.

FR19 (High Priority): The system shall allow a targeted employee to deny a shift swap request.

FR20 (High Priority): When a shift swap request is approved, the system shall give the shift from the requesting employee to the approving employee, and consequently update the status to be approved.

FR21 (Medium Priority): The system shall provide calendar shift data for the logged-in employee through an API endpoint so the weekly schedule can be up to date in the dashboard.

FR22 (High): The system shall provide an administrator dashboard that allows navigation to schedule management and availability management features.

FR23 (High): The system shall allow an administrator to view all employee schedules.

FR24 (High): The system shall allow an administrator to add a new shift for an employee.

FR25 (High): The system shall allow an administrator to edit an existing employee shift.

FR26 (High): The system shall allow an administrator to view all employee availability records.

FR27 (High): The system shall allow an administrator to edit an employee's availability.

FR28 (Medium): The system shall allow an administrator to navigate between scheduling and availability management features through a centralized admin dashboard.

3. Non-functional Requirements (10 points)

*List the **non-functional requirements** of the system (any requirement referring to a property of the system, such as security, safety, software quality, performance, reliability, etc.) You may provide a brief rationale for any requirement which you feel requires explanation as to how and/or why the requirement was derived.*

NFR1: The system shall securely store employee passwords using hashing rather than plaintext.

NFR2: The system shall restrict access to employee and administrator specific pages using session-based authentication.

NFR3: The system shall display clear error messages for invalid input, login failures, and system errors.

NFR4: The system shall maintain data integrity using a database with proper relationships between employees, shifts, availability, and swap requests.

4. Use Case Diagram (10 points)

*This section presents the **use case diagram** and the **textual descriptions** of the use cases for the system under development. The use case diagram should contain all the use cases and relationships between them needed to describe the functionality to be developed. If you discover new use cases between two increments, update the diagram for your future increments.*

Textual descriptions of use cases: For the first increment, the textual descriptions for the use cases are not required. However, the textual descriptions for all use cases discovered for your system are required for the second and third iterations.

1. View Dashboard

Primary Actor: Employee

Preconditions: The employee is logged into the system.

Main Flow: The employee selects or is redirected to the dashboard. The system retrieves the employee's information, including name, upcoming shifts, availability summary, submitted swap requests, and incoming targeted swap requests. The dashboard is then displayed.

Postconditions: The employee can view their current scheduling info and available actions from the dashboard.

2. View Upcoming Shifts

Primary Actor: Employee

Preconditions: The employee is logged into the system.

Main Flow: The employee accesses the dashboard. The system retrieves the employee's upcoming shifts from the database and displays them in the upcoming shifts section.

Postconditions: The employee can view their upcoming assigned shifts.

3. View Weekly Calendar

Primary Actor: Employee

Preconditions: The employee is logged into the system.

Main Flow: The employee accesses the dashboard. The system provides shift information for the employee and displays it in a weekly calendar view.

Postconditions: The employee can view their schedule in calendar format.

4. Add Availability

Primary Actor: Employee

Preconditions: The employee is logged into the system.

Main Flow: The employee navigates to the availability page, selects a day, start time, and end time, and submits the form. The system validates the input and stores the new availability block in the database.

Postconditions: A new availability entry is added for the employee.

5. Edit Availability

Primary Actor: Employee

Preconditions: The employee is logged into the system and already has at least one availability

entry.

Main Flow: The employee selects an existing availability block to edit. The system loads the current values. The employee updates the day and/or time values and submits the changes. The system validates the input and updates the record.

Postconditions: The selected availability entry is updated in the database.

6. Delete Availability

Primary Actor: Employee

Preconditions: The employee is logged into the system and already has at least one availability entry.

Main Flow: The employee selects an availability block to delete. The system verifies ownership of the record and removes it from the database.

Postconditions: The selected availability entry is deleted.

7. Submit Shift Swap Request

Primary Actor: Employee

Preconditions: The employee is logged into the system and has at least one upcoming shift.

Main Flow: The employee opens the shift swap request form from the dashboard, selects a shift, optionally enters a target employee ID and a reason, and submits the request. The system checks for a duplicate pending request and, if valid, stores the new request with pending status.

Postconditions: A new shift swap request is created and stored in the system.

8. Employee Login

Primary Actor: Employee

Preconditions: The employee has a valid employee ID stored in the system.

Main Flow: The employee enters their employee ID on the login page. The system checks whether the ID exists. If the employee already has a password, the system redirects them to the password entry page. If the employee does not yet have a password, the system redirects them to create one.

Postconditions: The employee is directed either to password entry or password creation.

9. Create Employee Password

Primary Actor: Employee

Preconditions: The employee has been identified in the system and does not yet have a password, or has been redirected to reset/create a password.

Main Flow: The employee enters a new password and confirms it. The system validates that both fields match and are not empty. The password is hashed and stored in the database. The system then clears the session and redirects the employee back to the login page.

Postconditions: The employee's password is securely stored in hashed form.

10. Logout

Primary Actor: Employee

Preconditions: The employee is currently logged into the system.

Main Flow: The employee clicks the logout option. The system clears the session and redirects the employee to the login page.

Postconditions: The employee is logged out of the system.

11. Cancel Pending Swap Request

Primary Actor: Employee

Preconditions: The employee is logged into the system and has at least one pending swap request they submitted.

Main Flow: From the employee's submitted swap requests, the employee selects a pending request to cancel. The system verifies that the request belongs to that employee and that its

status is still pending, then updates the request status to deleted.

Postconditions: The selected swap request is deleted.

12. Approve Shift Swap Request

Primary Actor: Employee

Preconditions: The employee is logged into the system and has at least one incoming pending shift swap request addressed to them.

Main Flow: The employee reviews incoming requests and chooses to approve one. The system verifies that the request is still pending and that the shift still belongs to the requester. The system updates the shift assignment to the approving employee and changes the request status to approved.

Postconditions: The shift is reassigned and the request is marked as approved.

13. Deny Shift Swap Request

Primary Actor: Employee

Preconditions: The employee is logged into the system and has at least one incoming pending shift swap request addressed to them.

Main Flow: The employee reviews incoming requests and chooses to deny one. The system verifies that the request is still pending and updates the request status to denied.

Postconditions: The request is marked as denied.

14. View My Swap Requests

Primary Actor: Employee

Preconditions: The employee is logged into the system.

Main Flow: The employee views the submitted swap requests section from the dashboard. The system retrieves the employee's submitted shift swap requests and displays their details and current statuses.

Postconditions: The employee can review all of their submitted swap requests.

15. Review Incoming Swap Requests

Primary Actor: Employee

Preconditions: The employee is logged into the system and is the target employee for one or more pending shift swap requests.

Main Flow: The employee opens the incoming swap requests section from the dashboard. The system retrieves all pending requests targeted to that employee and displays them for review. The employee may then decide whether to approve or deny a request.

Postconditions: The employee can review incoming swap requests and take action on them.

16. Admin Login

Primary Actor: Administrator

Preconditions: The administrator has a valid admin ID stored in the system.

Main Flow: The administrator enters the admin ID on the admin login page. The system checks whether the ID exists and then continues the login process by requiring the administrator to enter the admin password.

Postconditions: The administrator proceeds to the admin password step of authentication.

17. Enter Admin Password

Primary Actor: Administrator

Preconditions: The administrator has already entered a valid admin ID.

Main Flow: The administrator enters the password for the admin account. The system validates the entered password. If the password is correct, the administrator is redirected to the admin dashboard.

Postconditions: The administrator is authenticated and allowed to access admin features.

18. View Admin Dashboard

Primary Actor: Administrator

Preconditions: The administrator is logged into the system.

Main Flow: The administrator accesses the admin dashboard. The system displays the dashboard interface and available admin actions.

Postconditions: The administrator can view and access available admin dashboard features

19. View Employee Schedule (Admin)

Primary Actor: Administrator

Preconditions: The administrator is logged into the system.

Main Flow: The administrator selects view schedule and the system displays all employee shifts.

Postconditions: The administrator can view and access the schedules.

20. Add Shift

Primary Actor: Administrator

Preconditions: The administrator is logged into the system.

Main Flow: The administrator inputs employee, date, time. The system validates and saves.

Postconditions: A new shift is created.

21. Edit Shift

Primary Actor: Administrator

Preconditions: The administrator is logged into the system.

Main Flow: The administrator selects the shift and edits it

Postconditions: The shift is updated.

22. View Employee Availability (Admin)

Primary Actor: Administrator

Preconditions: The administrator is logged into the system.

Main Flow: The administrator selects the availability view and the system shows all.

Postconditions: The administrator can now see availability.

23. Edit Employee Availability (Admin)

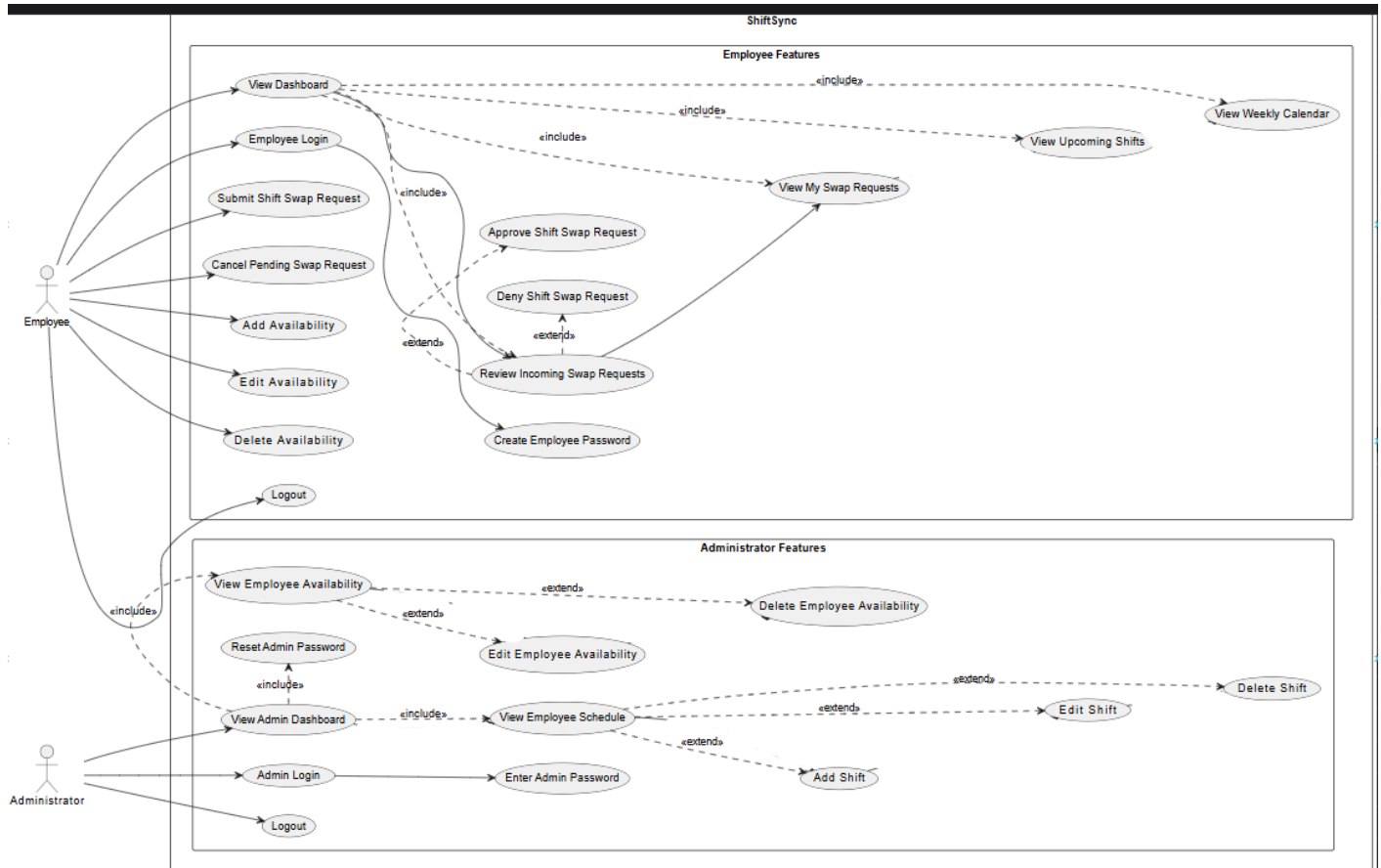
Primary Actor: Administrator

Preconditions: The administrator is logged into the system.

Main Flow: The administrator edits the selected availability and the system updates.

Postconditions: The employee availability is now updated.

.



5. Class Diagram and/or Sequence Diagrams (15 points)

This section presents a high-level overview of the anticipated system architecture using a **class diagram** and/or **sequence diagrams**.

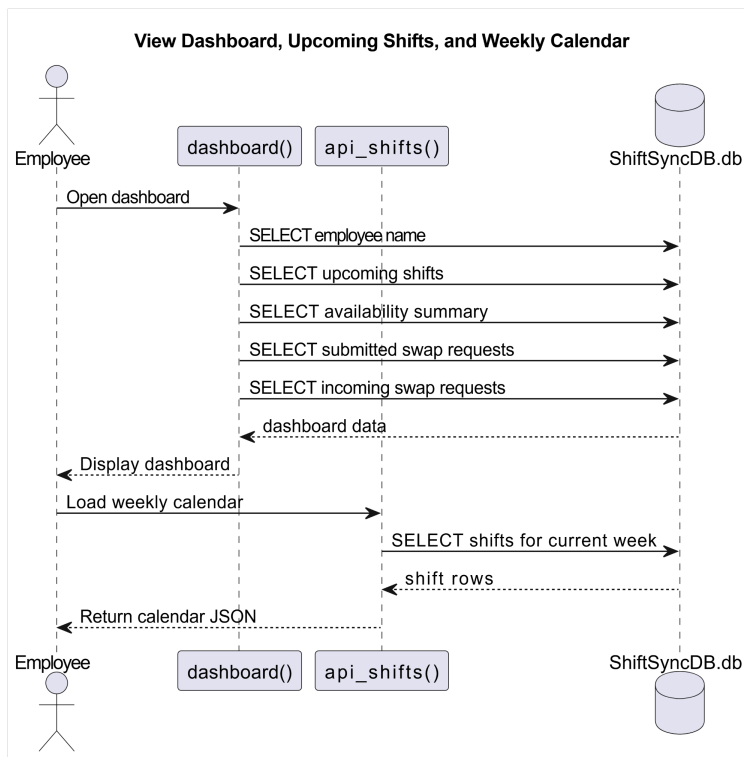
If the main **paradigm** used in your project is **Object Oriented** (i.e., you have classes or something that acts similar to classes in your system), then draw the **Class Diagram** of the **entire system** and **Sequence Diagrams** for the **three (3) most important use cases** in your system.

If the main **paradigm** in your system is **not Object Oriented** (i.e., you **do not** have classes or anything similar to classes in your system) then only draw **Sequence Diagrams**, but for **all the use cases of your system**. In this case, we will use a modified version of Sequence Diagrams, where instead of objects, the lifelines will represent the functions in the system involved in the action sequence.

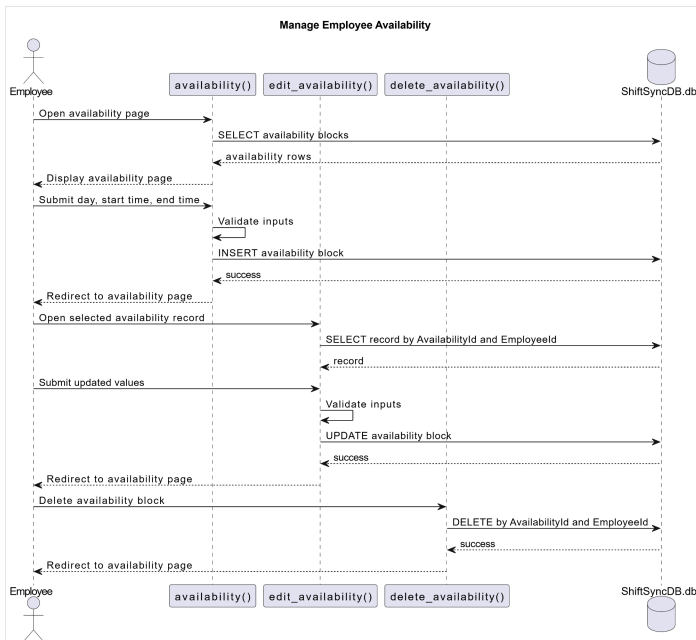
Class Diagrams show the **fundamental objects/classes** that must be modeled with the system to satisfy its requirements and **the relationships** between them. Each class rectangle on the diagram **must also include the attributes and the methods of the class** (they can be refined between increments). All the **relationships between classes and their multiplicity** must be shown on the class diagram.

A **Sequence Diagram** simply depicts **interaction between objects** (or **functions** - in our case - for non-OOP systems) in a sequential order, i.e. the order in which these interactions take place. Sequence diagrams describe how and in what order the objects in a system function.

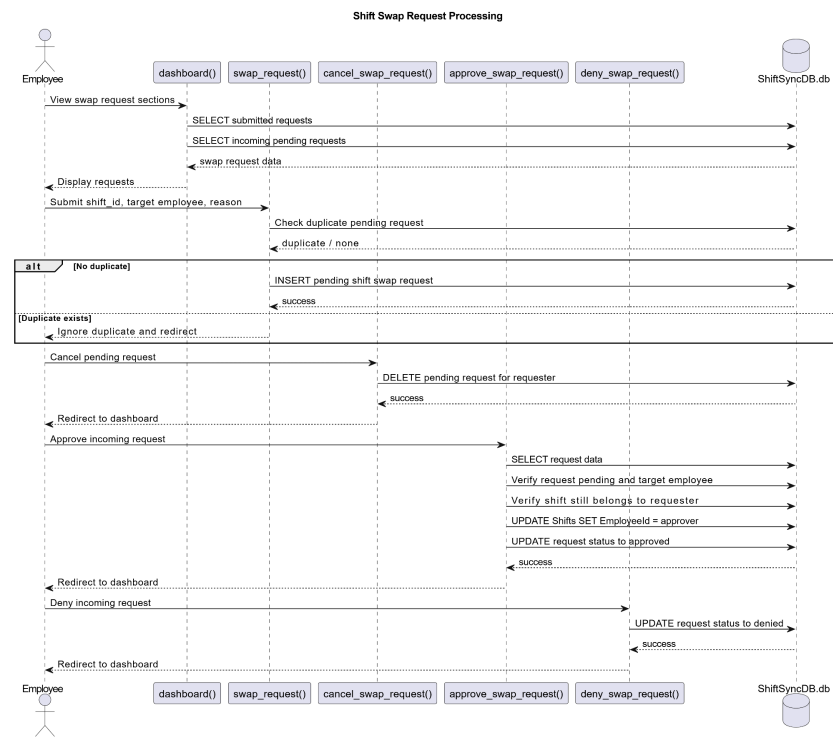
Sequence Diagram for Use Cases 1-3



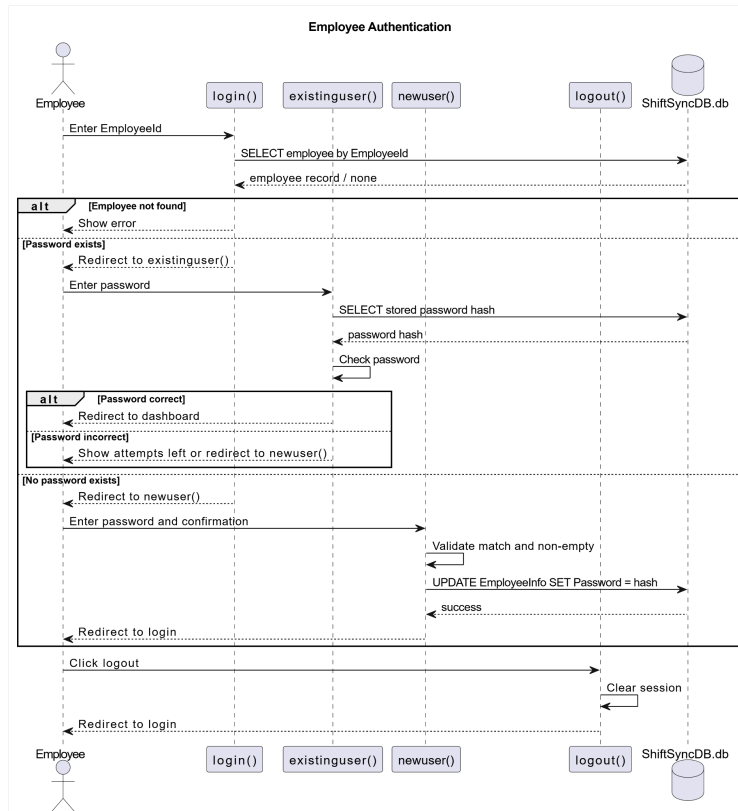
Sequence Diagram for Use Cases 4-6



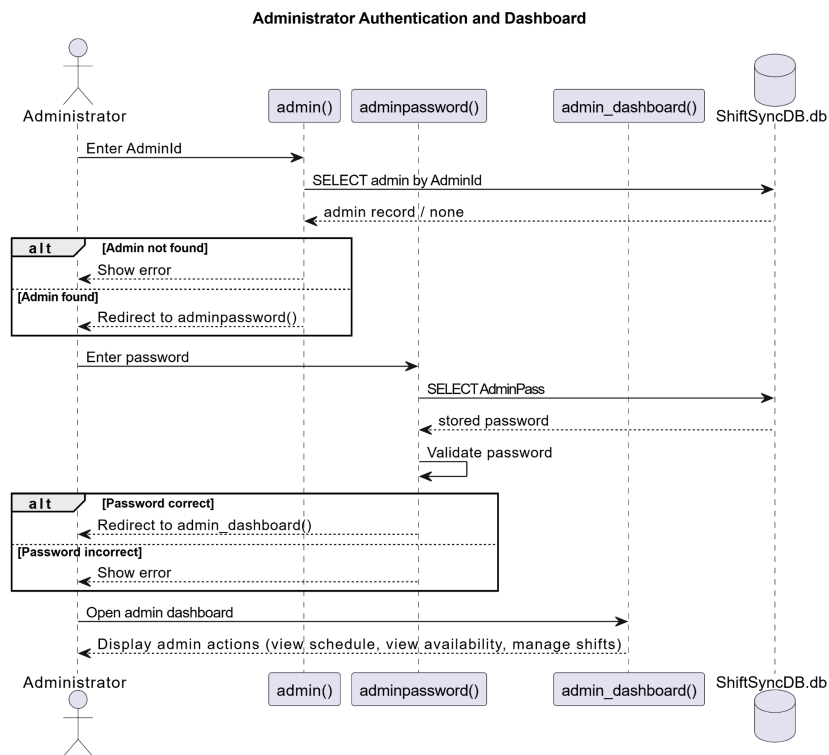
Sequence Diagram for Use Cases 7, 11-15



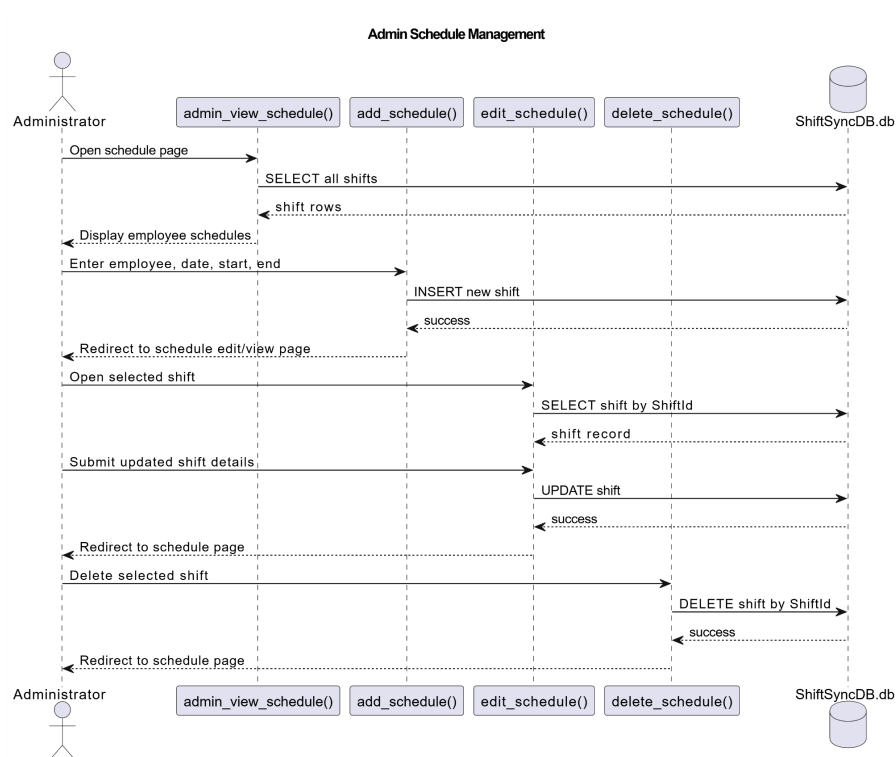
Sequence Diagram for Use Cases 8-10



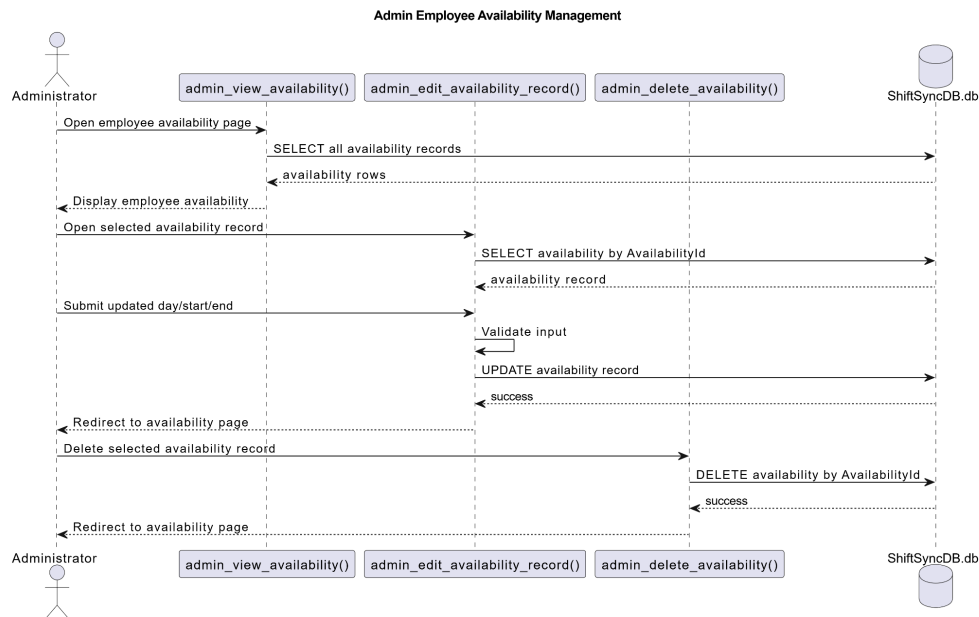
Sequence Diagram for Use Cases 16-18



Sequence Diagram for Use Cases 19-21



Sequence Diagram for Use Cases 22-23



6. Operating Environment (5 points)

Describe the environment in which the software will operate, including the hardware platform, operating system and versions, and any other software components or applications with which it must peacefully coexist.

ShiftSync operates in a web-based environment and is accessed in a browser. The application is developed with Python and Flask and can run on Windows, macOS, and Linux. It uses SQLite as database storage and an internet connection is needed.

7. Assumptions and Dependencies (5 points)

List any assumed factors (as opposed to known facts) that could affect the requirements stated in this document. These could include third-party or commercial components that you plan to use, issues around the development or operating environment, or constraints. The project could be affected if these assumptions are incorrect, are not shared, or change. Also identify any dependencies the project has on external factors, such as software components that you intend to reuse from another project.

- **ASSUMPTIONS:** Users should have internet connectivity available, Flask & SQLite remain supported
- **DEPENDENCIES:** Flask framework, SQLite database, Python runtime